

함수 — 단일행 함수 vs 그룹 함수

ch04.sql 코드를 실행 순서대로 분석한 학습자료입니다. HR 스키마(EMPLOYEES) 기준입니다.

01 단일행 함수 · DUAL 테이블

DUAL 은 오라클이 제공하는 1행짜리 가상 테이블로, 테이블 없이 함수/상수 계산 결과만 확인할 때 FROM절에 형식상 붙입니다.

```
SELECT sysdate FROM dual;
```

단일행 함수란 — 입력 행 하나당 결과도 하나(1:1)인 함수입니다. SYSDATE, TO_CHAR, NVL 등이 해당하며, 다음의 그룹함수(N:1)와 대비됩니다.

02 형변환 — 자동(암시적) vs 수동(명시적)

```
-- 자동 타입변환: 문자 '2'가 숫자로 암시적 변환됨
SELECT 1 + '2' FROM dual; -- 결과: 3
```

```
-- 수동 타입변환: TO_CHAR로 날짜 서식 지정
SELECT
  to_char(sysdate, 'YY'),
  to_char(sysdate, 'YYYY'),
  to_char(sysdate, 'MM'),
  to_char(sysdate, 'HH:MI:SS PM')
FROM dual;
```

```
SELECT TO_NUMBER('123') + TO_NUMBER('123') FROM dual; -- 결과: 246
```

실무 주의 — 암시적 형변환은 편리하지만 예상치 못한 결과나 인덱스 미사용(성능저하)의 원인이 됩니다. 조건절에서는 TO_CHAR/TO_NUMBER/TO_DATE로 명시적 변환을 권장합니다.

NVL(컬럼, 대체값) 은 컬럼 값이 NULL일 때만 지정한 대체값으로 바꿔줍니다. NULL이 포함된 연산·집계는 예기치 않게 NULL이 되거나 누락되므로 방어적으로 사용합니다.

```
-- commission_pct가 NULL이면 1(=100%)로 대체 후 곱셈 → NULL 때문에 계산이 날아가는 것
SELECT salary * nvl(commission_pct, 1)
FROM employees
ORDER BY commission_pct;
```

```
-- COUNT(컬럼)은 NULL을 세지 않는다 → NVL로 NULL을 값으로 바꿔야 전체 카운트 가능
SELECT COUNT(nvl(commission_pct, 1))
FROM employees;
```

핵심 — COUNT(*)는 행 개수 전체를 세지만, COUNT(컬럼명)은 그 컬럼이 NULL이 아닌 행만 셉니다. commission_pct가 NULL인 직원은 원래 COUNT(commission_pct)에서 빠지는데, NVL로 감싸면 전체가 다 카운트됩니다.

DECODE — Oracle 전용 함수. IF-ELSE와 유사하지만 "같음(=)" 비교만 가능.

DECODE(컬럼, 비교값, 결과값, 기본값)

```
-- department_id=60인 직원만 급여 10% 인상
SELECT first_name, last_name, department_id,
       salary 원래급여,
       decode(department_id, 60, salary*1.1, salary),
       decode(department_id, 60, '10% 인상', '동결')
FROM employees
ORDER BY department_id;
```

CASE WHEN — ANSI 표준. 범위·부등호 등 복잡한 다중 조건에 사용.

CASE WHEN 조건1 THEN 값1
WHEN 조건2 THEN 값2 ELSE 값3 END

```
-- 9000이상 상위 / 6000이상 중위 / 나머지 하위
SELECT first_name, last_name,
       CASE
         WHEN salary >= 9000 THEN '상위급여'
         WHEN salary >= 6000 THEN '중위급여'
         ELSE '하위급여'
       END AS 급여등급
FROM employees
WHERE job_id = 'IT_PROG';
```

선택 기준 — 단순 "같음" 매칭이면 DECODE가 짧고, >= 같은 범위/부등호 조건이 필요하면 반드시 CASE WHEN을 씁니다. DECODE는 조건식(>, <, BETWEEN 등)을 표현할 수 없습니다.

```
-- ROWNUM: 정렬과 무관하게 "조회된 순서"대로 매겨지는 가상번호 (정렬 후 번호가 아님에 주의)
SELECT ROWNUM AS 가상번호, employee_id, salary, first_name
FROM employees
ORDER BY 3 DESC;
```

```
-- 윈도우 함수: 정렬 기준에 따른 "진짜 순위"를 매김
SELECT employee_id, salary,
       RANK() OVER(ORDER BY salary DESC) rank_급여,
       DENSE_RANK() OVER(ORDER BY salary DESC) dense_rank_급여,
       ROW_NUMBER() OVER(ORDER BY salary DESC) row_number_급여
FROM employees;
```

함수	동점 처리	다음 순위
<code>RANK()</code>	동점에 같은 순위	동점 개수만큼 건너뛴 (1,1,3)
<code>DENSE_RANK()</code>	동점에 같은 순위	건너뛰지 않음 (1,1,2)
<code>ROW_NUMBER()</code>	무조건 순차 번호	동점이어도 1,2,3

ROWNUM 합정 — ORDER BY는 결과를 정렬해서 "보여줄 뿐", ROWNUM은 **정렬 이전** 원본 조회 순서 기준으로 이미 부여되어 있을 수 있습니다. "정렬된 순번"이 필요하다면 ROWNUM이 아니라 ROW_NUMBER() OVER(ORDER BY ...)를 써야 합니다.

그룹함수(집계함수)는 여러 행(N) → 하나의 결과(1)로 압축합니다. **GROUP BY** 는 특정 컬럼 값별로 그룹을 나눈 뒤 그 안에서 집계합니다.

```
SELECT COUNT(salary) 행수 FROM employees;
```

```
SELECT to_char(SUM(salary), '999,999,999.99') 합계,  
       to_char(AVG(salary), '999,999,999.99') 평균,  
       to_char(SUM(salary)/COUNT(salary), '999,999,999.99') 계산된평균  
FROM employees;
```

```
SELECT MAX(salary) 최대값, MIN(salary) 최소값, MAX(first_name), MIN(first_name)  
FROM employees;
```

문자열 MAX/MIN — 숫자뿐 아니라 문자열에도 사용 가능하며, 이 경우 사전순(알파벳순) 최대/최소를 반환합니다.

최고/최저 급여자의 이름 찾기 — 2가지 접근

```
-- 방법1: 서브쿼리로 최대/최소값을 구해 WHERE에 재사용  
select first_name, salary  
from employees  
where salary = (select max(salary) from employees)  
       or salary = (select min(salary) from employees);  
  
-- 방법2: 윈도우 함수 RANK로 최고/최저 순위(=1)인 행만 필터 (인라인뷰 활용)  
SELECT 성명, salary  
FROM (  
    SELECT first_name||' '||last_name AS 성명, salary,  
           RANK() OVER (ORDER BY salary ASC) AS 최저순위,  
           RANK() OVER (ORDER BY salary DESC) AS 최고순위  
    FROM employees  
)  
WHERE 최고순위 = 1 OR 최저순위 = 1;
```

GROUP BY — 단일 / 다중 그룹

```
-- job_id별 총급여·평균급여·인원수 (총급여 내림차순)
select job_id, sum(salary) 직급별_총급여, avg(salary) 직급별_평균, count(job_id) 인
from employees
group by job_id
order by 2 desc;

-- 다중 컬럼 그룹화: job_id + manager_id 조합별로 집계
select job_id, manager_id, sum(salary), avg(salary)
from employees
group by job_id, manager_id
order by job_id, manager_id;
```

GROUP BY 규칙 — SELECT절에 집계함수가 아닌 일반 컬럼이 있다면, 그 컬럼은 반드시 GROUP BY에도 포함되어야 합니다(그렇지 않으면 오류).

07 HAVING — GROUP BY 결과에 대한 조건절

WHERE 는 그룹화 이전 개별 행을 거르고, **HAVING** 은 그룹화·집계 이후 결과를 거릅니다. 집계함수 (SUM, AVG 등)에 조건을 걸려면 반드시 HAVING을 사용해야 합니다.

```
-- employee_id 10이상 대상(WHERE) → job_id별 평균급여 산출(GROUP BY)
--   → 평균급여 10000이상만(HAVING) → job_id 정렬
select job_id, avg(salary)
from employees
where employee_id >= 10
group by job_id
having avg(salary) >= 10000
order by 1;
```

실행 순서 — FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY 순으로 처리됩니다. HAVING이 GROUP BY 다음이라는 점, WHERE에는 집계함수를 못 쓴다는 점이 시험/실무에서 자주 나오는 함정입니다.